

ABLK: A Hybrid BB84 QKD and Geometric Indexing Framework for Secure Local Key Registries

Parth Anantrao Gawande

Abstract—This paper presents a local key-registry framework that combines a software simulation of Bennett–Brassard 1984 (BB84) key establishment with a geometric indexing mechanism, termed Angular-Bucket Lattice Key Mapping (ABLK). The indexing layer maps deterministic identifiers to coordinates in a bounded integer grid and assigns records to angular buckets relative to a secret reference coordinate. HMAC-SHA256 is used for lookup tags, while HKDF-SHA256 and AES-256-GCM are used for key derivation and authenticated encryption of stored values. A reproducibility benchmark uses 100 measured trials and 10 warm-up trials and compares the implemented registry against an authenticated dictionary baseline. In the tested three-dimensional configuration, the reference coordinate occupies 24 bytes. The results show that the geometric mapping itself is inexpensive, but the complete ABLK store and retrieval path is slower than the conventional dictionary baseline for the tested single-record workload. A size comparison with ML-KEM-1024 is included only as a representation-size reference; the two mechanisms perform different cryptographic functions and are not security-equivalent. The evaluation therefore focuses on local indexing and integration cost rather than on a new post-quantum cryptographic primitive.

Index Terms—Post-Quantum Cryptography, Quantum Key Distribution, BB84, Key Management, Geometric Indexing, AES-GCM, HMAC, ML-KEM.

I. INTRODUCTION

The emergence of large-scale quantum computing is expected to affect public-key cryptographic mechanisms based on integer factorization and discrete logarithms. Shor’s algorithm provides polynomial-time quantum algorithms for these problems, creating a long-term motivation for migration toward quantum-resistant cryptographic mechanisms [1]. In response, post-quantum cryptography (PQC) develops computational cryptographic schemes intended to remain secure against adversaries equipped with quantum computers. In parallel, Quantum Key Distribution (QKD) provides a physical-layer approach for establishing symmetric keys using the properties of quantum states [2].

Although key establishment receives substantial attention, locally stored key material also represents an important component of the overall key lifecycle. Once a session key has been established, an endpoint may need to store encrypted key material and retrieve it repeatedly during application operation. The organization of such local registries affects storage consumption, lookup latency, and exposure of metadata.

NIST standardized ML-KEM in Federal Information Processing Standard (FIPS) 203 as a post-quantum key-encapsulation mechanism derived from CRYSTALS-Kyber [3]. ML-KEM provides a standardized mechanism for establishing shared secrets over public channels. Its parameter sets are designed around structured lattice problems and provide substantially different security and performance characteristics from a lightweight local indexing structure.

This paper investigates a complementary approach for local key registries. Instead of proposing a replacement for a standardized KEM, the proposed Angular-Bucket Lattice Key Mapping (ABLK) mechanism focuses on the organization and retrieval of already established key material. A simulated BB84 phase establishes a symmetric session key, which is then used to protect a secret geometric reference coordinate. Local registry entries are mapped to deterministic grid coordinates and assigned to angular buckets relative to this reference coordinate.

The objective is not to replace ML-KEM or another standardized PQC mechanism. Instead, the study examines whether a geometric index can organize local registry entries with modest storage and lookup overhead while conventional authenticated encryption protects the stored values.

The main contributions of this work are as follows:

- **Hybrid Key-Management Architecture:** A framework combining simulated BB84 key establishment with a geometric local key-indexing mechanism is presented.
- **Geometric Registry Indexing:** A deterministic mapping from hashed key identifiers to bounded multidimensional coordinates and angular buckets is developed.
- **Authenticated Local Storage:** HMAC-SHA256, HKDF-SHA256, and AES-GCM are integrated to provide authenticated lookup tags, key derivation, and confidentiality with integrity for stored values.
- **Experimental Evaluation:** A reproducibility benchmark measures BB84 simulation, reference-coordinate protection, ABLK mapping, storage, and retrieval over 100 trials and compares the implemented registry against an authenticated dictionary baseline.
- **Security Scope Clarification:** The paper distinguishes the security provided by established cryptographic primitives from the security properties of the proposed geometric indexing structure.

II. BACKGROUND AND SYSTEM MODEL

A. Quantum Key Distribution

BB84 is a quantum key-distribution protocol introduced by Bennett and Brassard in 1984 [2]. The protocol uses two incompatible measurement bases to establish correlated random bits between two parties. After quantum transmission, the communicating parties publicly compare their selected bases and retain only measurements obtained using compatible bases.

In this work, BB84 is simulated rather than implemented using physical optical equipment. The simulation produces a raw key, followed by basis sifting and a hash-based key derivation step. Consequently, the experimental results do not represent the transmission characteristics, optical losses, detector noise, or security guarantees of a physical QKD deployment.

B. Post-Quantum Key Establishment

NIST FIPS 203 specifies the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), derived from CRYSTALS-Kyber [3]. ML-KEM is designed for establishing a shared secret over a public communication channel and is based on the Module Learning With Errors problem.

ML-KEM is used in this work only as a reference point for parameter-size comparison. ABLKM does not implement ML-KEM and does not claim to provide equivalent cryptographic security.

C. Geometric Indexing Model

Let the registry operate over a bounded integer grid

$$\Lambda_N = \{(x_1, \dots, x_d) \mid x_i \in \{0, \dots, N-1\}\}. \quad (1)$$

The number of possible grid points is therefore

$$M = N^d. \quad (2)$$

A secret reference coordinate is defined as

$$P_{\text{ref}} = (r_1, r_2, \dots, r_d) \in \mathbb{R}^d. \quad (3)$$

The reference coordinate is selected near the center of the grid. Let

$$c = \left(\frac{N-1}{2}, \frac{N-1}{2}, \dots, \frac{N-1}{2} \right). \quad (4)$$

Each coordinate is sampled according to

$$r_i \sim U(c_i - \delta N, c_i + \delta N), \quad (5)$$

where δ controls the sampling range.

The implementation rejects a candidate reference coordinate if it is numerically too close to an integer grid point:

$$\min_{p \in \Lambda_N} \|P_{\text{ref}} - p\|_2 > \epsilon, \quad (6)$$

where $\epsilon = 10^{-9}$.

III. PROPOSED ABLKM FRAMEWORK

The proposed system contains three logical phases:

- 1) simulated BB84 key establishment;
- 2) protected exchange of the geometric reference coordinate;
- 3) local storage and retrieval using geometric indexing.

The geometric mechanism is referred to as Angular-Bucket Lattice Key Mapping (ABLKM).

A. Parameter Setup

The setup procedure initializes the secret reference coordinate, grid parameters, angular buckets, and local database.

Algorithm 1 ABLKM Parameter Setup

Require: Grid limit N , dimension d , offset factor δ

Ensure: P_{ref} , bucket width W_θ , bucket count B_{count} , database DB

- 1: Compute centroid $c \leftarrow ((N-1)/2, \dots, (N-1)/2)$
 - 2: **repeat**
 - 3: Sample P_{ref} using $r_i \sim U(c_i - \delta N, c_i + \delta N)$
 - 4: **until** $\min_{p \in \Lambda_N} \|P_{\text{ref}} - p\|_2 > 10^{-9}$
 - 5: Compute $M \leftarrow N^d$
 - 6: Select target bucket density max_density
 - 7: Compute $B_{\text{count}} \leftarrow \lceil M / \text{max_density} \rceil$
 - 8: Compute $W_\theta \leftarrow 2\pi / B_{\text{count}}$
 - 9: Initialize DB with B_{count} buckets
 - 10: **return** $P_{\text{ref}}, W_\theta, B_{\text{count}}, DB$
-

B. Record Insertion

For a key identifier Key , SHA-256 is first used to produce a deterministic integer index. The index is then converted into a multidimensional coordinate.

The displacement from the secret reference coordinate is

$$v = \text{coord}_{\text{pt}} - P_{\text{ref}}. \quad (7)$$

For computational efficiency, the current implementation uses the first two coordinates of v to calculate the azimuth angle:

$$\theta = \text{atan2}(v_2, v_1) \pmod{2\pi}. \quad (8)$$

The resulting angle determines the bucket:

$$b = \min \left(\left\lfloor \frac{\theta}{W_\theta} \right\rfloor, B_{\text{count}} - 1 \right). \quad (9)$$

The value itself is protected independently of the geometric index. A lookup tag is generated using HMAC-SHA256, and an encryption key is derived using HKDF-SHA256. AES-GCM is then used to provide authenticated encryption.

C. Record Retrieval

The retrieval process repeats the deterministic coordinate and angular calculations. The lookup tag is recomputed and compared with the stored tag. If a matching record is found, the corresponding salt is used to derive the AES-GCM encryption key.

Algorithm 2 ABLKM Record Insertion**Require:** P_{ref} , key identifier Key , value Val , database DB **Ensure:** Encrypted registry entry

- 1: $pt_{\text{idx}} \leftarrow \text{SHA256}(Key) \bmod N^d$
- 2: **Map** pt_{idx} to coord_{pt}
- 3: $v \leftarrow \text{coord}_{\text{pt}} - P_{\text{ref}}$
- 4: $\theta \leftarrow \text{atan2}(v_2, v_1) \pmod{2\pi}$
- 5: $b \leftarrow \min(\lfloor \theta/W_\theta \rfloor, B_{\text{count}} - 1)$
- 6: **Generate** 32-byte salt and 12-byte nonce
- 7: $K_{\text{hmac}} \leftarrow \text{SHA256}(P_{\text{ref}})$
- 8: $\text{tag} \leftarrow \text{HMAC-SHA256}(K_{\text{hmac}}, Key)$
- 9: $Ikm \leftarrow \text{SHA256}(Key \parallel \text{coord}_{\text{pt}} \parallel P_{\text{ref}})$
- 10: $K_{\text{aes}} \leftarrow \text{HKDF-SHA256}(Ikm, \text{salt}, \text{info})$
- 11: $C \leftarrow \text{AES-GCM-Encrypt}(K_{\text{aes}}, \text{nonce}, Val)$
- 12: **Store** $\{\text{tag}, \text{nonce}, C, \text{salt}\}$ in B_b

Algorithm 3 ABLKM Record Retrieval**Require:** P_{ref} , key identifier Key , database DB **Ensure:** Decrypted value Val or failure

- 1: $pt_{\text{idx}} \leftarrow \text{SHA256}(Key) \bmod N^d$
- 2: **Map** pt_{idx} to coord_{pt}
- 3: $v \leftarrow \text{coord}_{\text{pt}} - P_{\text{ref}}$
- 4: $\theta \leftarrow \text{atan2}(v_2, v_1) \pmod{2\pi}$
- 5: $b \leftarrow \min(\lfloor \theta/W_\theta \rfloor, B_{\text{count}} - 1)$
- 6: $K_{\text{hmac}} \leftarrow \text{SHA256}(P_{\text{ref}})$
- 7: $\text{tag} \leftarrow \text{HMAC-SHA256}(K_{\text{hmac}}, Key)$
- 8: **Search** bucket B_b for matching tag
- 9: **if** matching entry is not found **then**
- 10: **return** Fail
- 11: **end if**
- 12: **Extract** $\{\text{nonce}, C, \text{salt}\}$
- 13: $Ikm \leftarrow \text{SHA256}(Key \parallel \text{coord}_{\text{pt}} \parallel P_{\text{ref}})$
- 14: $K_{\text{aes}} \leftarrow \text{HKDF-SHA256}(Ikm, \text{salt}, \text{info})$
- 15: $Val \leftarrow \text{AES-GCM-Decrypt}(K_{\text{aes}}, \text{nonce}, C)$
- 16: **return** Val

D. Coordinate Mapping

The integer index can be deterministically converted into d grid coordinates using

$$x_i = \left\lfloor \frac{\text{Idx}}{N^{i-1}} \right\rfloor \bmod N. \quad (10)$$

The mapping requires $O(d)$ arithmetic operations. Since the coordinate mapping is deterministic, the same key identifier produces the same grid coordinate during insertion and retrieval.

E. Metadata Protection

A plaintext lookup identifier could reveal information about key usage. ABLKM therefore uses an HMAC-derived lookup tag rather than storing the identifier directly.

The HMAC key is derived from the secret reference coordinate:

$$K_{\text{hmac}} = \text{SHA256}(P_{\text{ref}}), \quad (11)$$

and the lookup tag is

$$T_{\text{lookup}} = \text{HMAC-SHA256}(K_{\text{hmac}}, Key). \quad (12)$$

This construction provides authentication of lookup metadata under the assumption that P_{ref} remains secret. It should not, however, be interpreted as a substitute for a formally analyzed searchable-encryption construction.

IV. SIMULATED BB84 KEY COUPLING

The first phase of the prototype simulates the BB84 protocol using two measurement bases: rectilinear and diagonal. After basis reconciliation, nonmatching basis positions are discarded.

The resulting sifted sequence is transformed into a 256-bit session key using

$$K_{\text{QKD}} = \text{SHA256}(K_{\text{raw}}). \quad (13)$$

In the prototype, K_{QKD} is used to protect the transmission of P_{ref} over the simulated classical channel.

The implementation subsequently removes the QKD-derived session key from active memory. This behavior is intended to reduce the lifetime of the session key in the prototype; it should not be interpreted as a complete proof of forward secrecy, since forward secrecy depends on the complete key-establishment and session-management protocol.

V. SECURITY ANALYSIS**A. Threat Model**

The security analysis considers an adversary who may obtain a copy of the local registry and observe stored ciphertexts, salts, nonces, lookup tags, and bucket assignments. The adversary is assumed not to possess the secret reference coordinate or the active cryptographic keys.

The model does not assume that the geometric structure itself constitutes a post-quantum cryptographic primitive. Instead, confidentiality and integrity of stored values are provided by AES-GCM, while lookup metadata is protected using HMAC-SHA256.

The BB84 component is simulated and therefore does not provide a claim about the security of an actual optical QKD implementation.

B. Role of the Geometric Reference Coordinate

The reference coordinate acts as a secret indexing parameter. An attacker who does not know P_{ref} may have difficulty reproducing the exact bucket calculation. However, the current construction does not establish that recovering P_{ref} is equivalent to solving a standard hard lattice problem.

In particular, the implementation uses the identity basis $B = I_d$ over a bounded integer grid. Such a construction should not be treated as equivalent to the structured lattices used by standardized post-quantum schemes such as ML-KEM.

Consequently, no independent post-quantum security level is claimed for ABLKM.

C. Security Interpretation of the Geometric Parameter

The geometric construction uses a secret reference coordinate, but no claim is made that recovering this coordinate is equivalent to solving CVP, SVP, Module-LWE, or another recognized hard problem. The bounded integer grid and identity basis used by the implementation are indexing structures rather than a cryptographic lattice construction. Accordingly, the numerical range or decimal precision of P_{ref} is not treated as a cryptographic security parameter, and no brute-force parameter-count estimate is interpreted as a security level.

D. Authenticated Encryption

The confidentiality of stored values is provided by AES-GCM. The encryption key is derived from the key identifier, its deterministic coordinate, and the secret reference coordinate through SHA-256 and HKDF-SHA256.

The construction therefore separates two functions:

- geometric indexing determines where a record is located;
- authenticated encryption protects the contents of the record.

This separation is important because geometric indexing alone does not provide confidentiality.

E. Floating-Point Considerations

The angular bucket calculation depends on floating-point arithmetic. Let ϵ_f denote the numerical error associated with the implementation. For a point at distance r from the reference coordinate, a first-order approximation gives

$$|\Delta\theta| = O\left(\frac{\epsilon_f}{r}\right). \quad (14)$$

Consequently, points sufficiently close to a bucket boundary may be sensitive to numerical rounding. The prototype reduces this risk by rejecting reference coordinates that are too close to integer grid points, but this does not eliminate all possible boundary effects.

A production implementation should therefore include deterministic numerical tests and explicit handling for bucket-boundary conditions.

VI. EXPERIMENTAL EVALUATION

A. Experimental Environment

The benchmark is implemented in Python and uses the `cryptography` package. Each benchmark consists of 10 warm-up trials followed by 100 measured trials using `time.perf_counter_ns`. The reference configuration uses 1024 simulated BB84 qubits, a three-dimensional grid with $N = 6$, and four lattice points per angular bucket. The payload is a short synthetic string and no external database or network I/O is included in the timing.

The target experimental environment for the manuscript is a Windows 11 system with an Intel Core i7-1165G7 processor at 2.80 GHz and 16 GB RAM, using Python 3.11. The artifact also records the Python version, operating-system information, processor identifier when available, benchmark parameters, timer, and BB84-output suppression in

`benchmark_environment.txt`. The currently packaged benchmark table is a reference run executed in a separate Linux software environment with Python 3.13.5; its processor identifier was not exposed. Accordingly, the numerical values in the packaged CSV must not be interpreted as measurements from the i7-1165G7 system. For a final submission, the benchmark should be rerun on the stated Windows 11/i7-1165G7 machine and the resulting CSV and environment metadata should replace the reference-run values.

The BB84 component is a software simulation. It uses the implementation's basis selection, measurement, sifting, and SHA-256 derivation, but it does not model optical losses, detector imperfections, a physical authenticated classical channel, or a complete deployed BB84 post-processing stack. Console output is suppressed during timing so that printing is not included in the BB84 measurement.

B. Latency Evaluation

Table I reports the arithmetic mean and sample standard deviation over 100 measured trials. The protected reference-coordinate operation measures AES-GCM encryption followed by decryption using one BB84-derived key; it does not measure a physical key exchange. The ABLKM retrieval path follows the current implementation, which uses an HMAC-derived registry dictionary to obtain the bucket before authenticated decryption. Thus, the measurement does not claim a bucket-scan search over the geometric sectors.

TABLE I
REFERENCE-RUN EXECUTION TIMES OF THE IMPLEMENTED PROTOTYPE

Operation	Mean (ms)	S.D. (ms)
BB84 simulation + derivation	2.0781	0.9181
P_{ref} AES-GCM protection/recovery	0.0050	0.0129
ABLKM geometric mapping	0.0026	0.0072
ABLKM store	0.0650	0.0804
ABLKM retrieval	0.0214	0.0154
Baseline secure-dictionary store	0.0099	0.0024
Baseline secure-dictionary retrieval	0.0059	0.0033

The geometric mapping component is substantially smaller than the complete storage path. However, the complete ABLKM store and retrieval operations are slower than the authenticated dictionary baseline in this single-record workload. This is an important negative result: the current implementation does not demonstrate a universal latency advantage over a conventional local key-value index. The appropriate claim is instead that the geometric mapping can be added with small absolute computation cost while providing a different organization mechanism.

The reference-coordinate representation is compact, but representation size should not be confused with database storage cost or cryptographic security. Likewise, the measured BB84 latency is a software-simulation result and is not a physical QKD throughput measurement.

C. Baseline Definition

The baseline removes geometric mapping and uses a conventional dictionary keyed by the same HMAC-derived

identifier. It retains randomized salts, HKDF-SHA256, and AES-256-GCM so that the comparison remains a local storage/indexing comparison rather than a comparison between different cryptographic primitives. The baseline is not a cryptographic equivalent of ABLKM, and the experiment does not establish that one approach is superior for all registry sizes or workloads.

D. Storage Comparison

Table II compares the storage parameters of ML-KEM-1024 with the reference-coordinate representation used by the three-dimensional ABLKM prototype.

According to FIPS 203, ML-KEM-1024 has an encapsulation key of 1568 bytes, a decapsulation key of 3168 bytes, and a ciphertext of 1568 bytes [3].

For $d = 3$, storing three IEEE 754 double-precision coordinates requires

$$3 \times 8 = 24 \text{ bytes.} \quad (15)$$

TABLE II
STORAGE-SIZE REFERENCE COMPARISON

Parameter	ML-KEM-1024	ABLKM ($d = 3$)
Encapsulation/public parameter	1568 B	24 B
Private parameter	3168 B	–
Ciphertext	1568 B	–
Reference coordinate	–	24 B

The 24-byte reference coordinate is approximately 98.47% smaller than the 1568-byte ML-KEM-1024 encapsulation key:

$$1 - \frac{24}{1568} \approx 0.9847. \quad (16)$$

This percentage concerns only the size of the reference-coordinate representation. It excludes the HMAC tag, salt, nonce, ciphertext, database structures, and other implementation metadata. Moreover, the ABLKM reference coordinate is not a replacement for an ML-KEM encapsulation key; the two values serve different cryptographic functions.

E. Local Lookup Performance

The reference run shows that geometric mapping itself requires only a few microseconds, whereas the complete ABLKM store and retrieval paths include HMAC, random salt and nonce generation, HKDF derivation, AES-GCM processing, and registry bookkeeping. The authenticated dictionary baseline is faster for the tested single-record workload. This comparison is more informative than comparing the local registry with a KEM because the two perform different functions.

A more complete evaluation should vary registry size, record distribution, bucket occupancy, collision rate, and retrieval workload. In particular, future versions should implement and measure an explicit bucket-scan retrieval path if geometric bucketing is intended to serve as the primary search structure; the present implementation's direct HMAC registry makes retrieval effectively dictionary-based.

VII. DISCUSSION

The reference run indicates that geometric indexing can be computed at low absolute cost, but it does not outperform a conventional authenticated dictionary in the tested single-record workload. The principal observed property is therefore compact representation and deterministic geometric organization, not lower latency than conventional indexing.

The framework is intended as a complementary layer rather than a replacement for standardized post-quantum cryptography. In a practical deployment, a standardized mechanism such as ML-KEM could be used for network-level key establishment, while ABLKM could be used to organize encrypted key material after it reaches an endpoint.

The separation between key establishment, local indexing, and authenticated encryption also provides a modular architecture. The indexing mechanism can be changed without changing the underlying AES-GCM protection of stored values.

Several limitations remain. First, the current prototype uses only a two-dimensional angular projection even when the registry is represented in a higher-dimensional coordinate space. Second, the geometric parameter has not been reduced to a recognized cryptographic hardness assumption. Third, the security of the system depends on correct protection of the reference coordinate and cryptographic keys. Finally, the current evaluation uses synthetic workloads and a software BB84 simulation rather than physical QKD equipment.

VIII. LIMITATIONS AND FUTURE WORK

The following directions are proposed for future research:

- **Formal Security Analysis:** Develop a formal security model for the geometric indexing mechanism and determine whether any meaningful reduction to a recognized computational problem can be established.
- **Alternative Indexing Structures:** Compare angular bucketing against conventional hash tables, balanced trees, encrypted indexes, and other local registry structures under identical workloads.
- **Higher-Dimensional Evaluation:** Investigate whether using additional coordinate dimensions improves distribution or security without increasing lookup overhead significantly.
- **Side-Channel Evaluation:** Study timing, cache, power, and electromagnetic leakage associated with coordinate calculations and cryptographic operations.
- **Physical QKD Integration:** Replace the simulated BB84 component with an optical QKD implementation to evaluate realistic communication and synchronization overhead.
- **Hardware Acceleration:** Investigate FPGA or embedded implementations for deterministic coordinate mapping and cryptographic operations.
- **Large-Scale Database Testing:** Evaluate the approach with substantially larger registries and different bucket-density configurations to determine scalability.

IX. REPRODUCIBILITY AND ARTIFACT AVAILABILITY

The benchmark scripts and source implementation are intended to support independent reproduction of the reported

software measurements. The artifact includes the BB84 simulation, ABLKM implementation, hybrid integration test, benchmark script, CSV output, and environment metadata. The benchmark does not require an external dataset because the workload is synthetically generated. To reproduce the reference run, install the required Python dependencies, run the benchmark script, and preserve the resulting CSV and environment metadata together with the manuscript version.

For a submission-specific performance claim, the benchmark should be rerun on the hardware and operating system used for the final experimental statement, and the resulting environment metadata should replace the reference-run description above. IEEE guidance emphasizes detailed methodology and sharing of code and research outputs to support independent verification and reproducibility.

X. CONCLUSION

This paper presented a local key-management framework that combines a simulated BB84 key-establishment phase with Angular-Bucket Lattice Key Mapping (ABLKM). The geometric layer uses a secret reference coordinate to organize registry entries into angular buckets, while HMAC-SHA256, HKDF-SHA256, and AES-GCM protect lookup metadata and stored values.

The reference implementation was evaluated over 100 measured trials after 10 warm-up trials. In the tested three-dimensional configuration, the geometric mapping required only a few microseconds, while the complete ABLKM store and retrieval paths were slower than the authenticated dictionary baseline for the single-record workload. The three-coordinate reference representation requires 24 bytes, but this is only a representation-size observation and excludes associated registry metadata.

The prototype demonstrates that the proposed mapping can be implemented with low absolute computational cost, but it does not demonstrate a latency advantage over conventional local indexing. This result should be interpreted as an evaluation of an indexing layer, not as evidence for a new cryptographic security level. The integer grid and angular bucketing do not establish security equivalent to a lattice-based scheme such as ML-KEM. ABLKM is therefore best regarded as a local key-management and indexing component that can operate alongside established key-establishment and encryption mechanisms.

Future work will focus on formal security analysis, large-scale evaluation, side-channel testing, and integration with physical QKD systems.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci. (FOCS)*, Santa Fe, NM, USA, 1994, pp. 124–134.
- [2] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," in *Proc. IEEE Int. Conf. Comput., Syst. Signal Process.*, Bangalore, India, 1984, pp. 175–179.
- [3] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," Federal Information Processing Standards Publication (FIPS) 203, Aug. 2024.
- [4] Y. Chen and P. Q. Nguyen, "BKZ 2.0: Better lattice security estimates," in *Advances in Cryptology – ASIACRYPT 2011*, LNCS, vol. 7073, Springer, 2011, pp. 1–20.
- [5] D. Micciancio and S. Goldwasser, *Complexity of Lattice Problems: A Cryptographic Perspective*. Boston, MA, USA: Kluwer Academic Publishers, 2002.
- [6] J. Hoffstein, J. Pipher, and J. H. Silverman, "NTRU: A ring-based public key cryptosystem," in *Algorithmic Number Theory (ANTS-III)*, LNCS, vol. 1423, Springer, 1998, pp. 267–288.
- [7] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, pp. 1–40, 2009.